

Chapter 3: Search Techniques

5 hours | 9 marks

"The right algorithm is the difference between waiting your entire lifetime for an answer and getting it in seconds."

— Steven Skiena

The Hook — Why Do We Need Search?

You've got the state space (Chapter 2). Now: **how do you find the path through it?**

Imagine a GPS trying to route from Kathmandu to Pokhara. There are hundreds of possible routes. How does it find the best one without checking every single combination?

The answer: **intelligent search** — the core of classical AI.

In this chapter, we'll move through **three generations** of search:

1. **Uninformed Search** — brute force, no map
2. **Informed Search** — uses hints to be smarter
3. **Adversarial Search** — for games with an opponent

Part 1: Uninformed (Blind) Search

"I don't know which direction the goal is. I'll explore systematically."

Uninformed search means the algorithm only knows:

- The current state
- The available actions
- Whether it's at the goal

It has **no "sense of direction"** — no idea whether it's getting closer or farther.

3.1 Depth-First Search (DFS)

The Idea

"Go as deep as possible along one path. Backtrack only when stuck."

Think of exploring a maze by always turning left — you go deep until you hit a dead end, then backtrack.

How It Works

```
  A
 /|\
B C D
 /| |
E F G
  |
  H ← GOAL
```

DFS explores in this order: $A \rightarrow B \rightarrow E \rightarrow (\text{backtrack}) \rightarrow F \rightarrow (\text{backtrack}) \rightarrow C \rightarrow (\text{backtrack}) \rightarrow D \rightarrow G \rightarrow (\text{backtrack}) \rightarrow H$ ✓

DFS uses a **Stack** (Last-In, First-Out).

```
Initial Stack: [A]
Pop A, push children: [B, C, D]
Pop D (last in), push children: [B, C, G]
Pop G (dead end): [B, C]
Pop C (dead end): [B]
Pop B, push children: [E, F]
Pop F, push children: [H]
Pop H → GOAL! ✓
```

Properties

Property	DFS
Complete?	No (can get stuck in infinite loops)
Optimal?	No (might find a deep, non-optimal solution)
Time	$O(b^m)$ — b =branching factor, m =max depth
Space	$O(bm)$ ← Best feature of DFS! Linear space!

DFS shines when: Space is limited, solutions are deep, and you just need a solution (not necessarily optimal).

3.2 Breadth-First Search (BFS)

The Idea

"Explore all neighbors first, then their neighbors, then their neighbors..."

Like ripples spreading outward from a stone dropped in water — BFS expands level by level.

How It Works

```

Level 0:      A
              /|\
Level 1:      B C D
              /| |
Level 2:      E F G
              |
Level 3:      H ← GOAL
  
```

BFS explores in this order: $A \rightarrow B, C, D \rightarrow E, F, G \rightarrow H$ ✓

BFS uses a **Queue** (First-In, First-Out).

```

Queue: [A]
Deque A, enqueue children: [B, C, D]
Deque B, enqueue children: [C, D, E, F]
  
```

```

Dequeue C (no children): [D, E, F]
Dequeue D, enqueue children: [E, F, G]
Dequeue E (dead end): [F, G]
Dequeue F, enqueue children: [G, H]
Dequeue G (dead end): [H]
Dequeue H → GOAL! ✓

```

Properties

Property	BFS
Complete?	✓ Yes (if branching factor is finite)
Optimal?	✓ Yes (finds shallowest solution)
Time	$O(b^{(d+1)})$ — d =depth of solution
Space	$O(b^{(d+1)})$ ← Worst feature! Exponential space!

BFS shines when: You need the shortest-path solution and space is not an issue.

3.3 DFS vs BFS — Side-by-Side Comparison

BFS spreads like water:



DFS dives like a submarine:



Feature	DFS	BFS
Data Structure	Stack	Queue
Explores	Depth-first	Breadth-first
Memory	$O(bm)$ — linear	$O(b^d)$ — exponential

Feature	DFS	BFS
Time	$O(b^m)$	$O(b^d)$
Finds shortest path?	No	Yes
Complete?	Not always	Yes

3.4 Depth-Limited Search (DLS)

Problem with DFS: It can get stuck forever in infinite paths.

Solution: Impose a depth limit L .

If depth $> L$, don't go further.

DLS with limit $L=2$ on same tree:

```

      A
     /\
    B C D  ← Level 1 (explore)
   /\  |
  E F  G  ← Level 2 (explore)
   |
   H      ← Level 3 (CUT OFF — don't go here)

```

Problem: What if the solution is deeper than L ?

Solution: Iterative Deepening!

3.5 Iterative Deepening DFS (IDDFS)

"Best of both worlds: DFS's space, BFS's completeness."

Run DLS with $L=0$, then $L=1$, then $L=2...$ until goal found.

```

L=0: Try A → fail
L=1: Try A, B, C, D → fail
L=2: Try A, B, E, F, C, D, G → fail
L=3: Try A, B, E, F, H → FOUND! ✓

```

Yes, you re-explore earlier nodes. But the wasted work is small because deeper levels dominate.

Math insight: At depth d with branching factor b , total nodes explored $\approx b^d$
(The earlier levels are negligible in comparison!)

IDDFS Properties

Property	IDDFS
Complete?	✓ Yes
Optimal?	✓ Yes
Time	$O(b^d)$
Space	$O(bd)$ — linear!

IDDFS is often the best uninformed search. It's complete, optimal, and memory-efficient.

3.6 Strategy Comparison Table

Algorithm	Complete	Optimal	Time	Space
BFS	✓	✓ (uniform cost)	$O(b^d)$	$O(b^d)$
DFS	✗	✗	$O(b^m)$	$O(bm)$
DLS	✗	✗	$O(b^l)$	$O(bl)$
IDDFS	✓	✓	$O(b^d)$	$O(bd)$

b = branching factor, d = solution depth, m = max depth, l = depth limit

Part 2: Informed (Heuristic) Search

"I don't just wander — I use knowledge to guide my search toward the goal."

The key addition: a **heuristic function $h(n)$** — an estimate of how far node n is from the goal.

What makes a good heuristic?

- It's **admissible** — never overestimates the true cost (optimistic)
- It's **consistent** — satisfies the triangle inequality

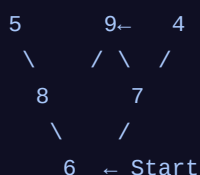
3.7 Hill Climbing

The Idea

**"Always move to the neighbor that looks best according to the heuristic.
Never go backward."**

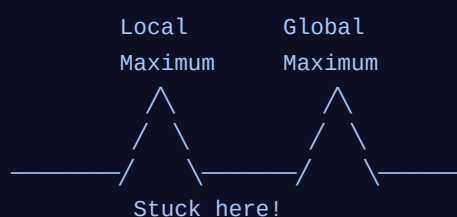
It's like climbing a mountain blindfolded — you always step uphill.

Value (higher = better):



From 6, move to 8 (highest neighbor). From 8, move to 9 (highest). 9 is a local maximum.

The Fundamental Problem: Local Maxima



Hill climbing gets **stuck at local maxima** — it thinks it's at the best point, but it's not.

Solutions:

- **Random Restart:** Start over from a random position
- **Simulated Annealing:** Sometimes accept worse moves (like cooling metal — high temp = random, low temp = precise)
- **Genetic Algorithms:** Maintain a population of solutions (Chapter 6)

3.8 Best-First Search

The Idea

"Always expand the node that seems closest to the goal."

Use a priority queue ordered by $h(n)$ (heuristic estimate to goal).

Greedy Best-First Search

Uses only $h(n)$ — ignores actual cost traveled.

Example: Romania Map Problem



We want to get from Arad to Bucharest.

Greedy uses $h(n)$ = straight-line distance to Bucharest:

- Arad: $h = 366$
- Sibiu: $h = 253$
- Fagaras: $h = 176$

- Bucharest: $h = 0$ ✓

Greedy quickly finds a path but **may not find the optimal one!**

3.9 A* Search — The Crown Jewel

The Idea

"Balance how far you've come with how far you still need to go."

$$f(n) = g(n) + h(n)$$

where:

$g(n)$ = actual cost from start to n (backward-looking)

$h(n)$ = estimated cost from n to goal (forward-looking)

$f(n)$ = estimated total cost of path through n

This is the most elegant and powerful idea in classical AI search.

Why This Works: The Intuition

Imagine you're racing to a destination:

- $g(n)$ = how much fuel you've burned
- $h(n)$ = GPS estimate of remaining fuel
- $f(n)$ = total estimated fuel for this route

You always take the route with the lowest estimated total fuel.

*A always expands nodes in order of $f(n)$.**

Worked Example: A* on Romania

Arad → Bucharest

Admissible Heuristic: Straight-line distance to Bucharest

Node	$g(n)$	$h(n)$	$f(n)$
Arad	0	366	366
Sibiu	140	253	393

```
Fagaras  239   176   415
Bucharest 450     0   450 ← GOAL!
```

A* finds the optimal route!

Why A* is Optimal

If $h(n)$ is **admissible** (never overestimates), A* is guaranteed to find the optimal path.

Proof intuition:

- If A* expands a non-optimal goal first, there must be a better path through the open list
- That better path has lower $f = \text{lower } g + \text{lower } h$
- But h never overestimates, so $f \leq \text{true cost}$
- Therefore A* would have expanded that better node first — contradiction!

Heuristics for Common Problems

8-puzzle heuristics:

Heuristic	Formula	Admissible?
h_1 (# misplaced tiles)	Count tiles not in goal position	✓ Yes
h_2 (Manhattan distance)	Σ	$\text{row}_i - \text{row}_{\text{goal}}$

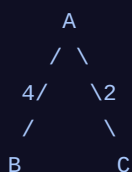
h_2 dominates h_1 : $h_2(n) \geq h_1(n)$ for all $n \rightarrow h_2$ is better!

Why Manhattan is admissible:

Each tile needs at least its Manhattan distance moves to reach its goal. So this is a lower bound. A lower bound = admissible.

3.10 Visual Comparison: All Informed Searches

Given this graph (numbers = edge costs):





h values (estimated distance to G):

$h(A)=6, h(B)=4, h(C)=3, h(D)=2, h(E)=1, h(G)=0$

Algorithm	Path Found	Total Cost	Optimal?
Greedy	$A \rightarrow C \rightarrow E \rightarrow G$	$2+1+3=6$	Maybe
A*	$A \rightarrow C \rightarrow E \rightarrow G$	6	✓ Yes

Part 3: Adversarial Search

"There is an opponent actively working against you. You cannot just find a path — you must anticipate their moves."

3.11 The Minimax Procedure

The Setup

Two players: **MAX** (you) and **MIN** (opponent).

- MAX wants to **maximize** the final score
- MIN wants to **minimize** the final score
- Both play **optimally**

The Game Tree

```

MAX's turn:      A
                  /|\
MIN's turn:      B C D

```

```

      /|  |  \
    3 5  2  9  4  ← Terminal states (utility values)

```

MIN plays optimally — MIN picks the minimum from each of its subtrees:

- B's children: {3, 5} → MIN picks 3
- C's children: {2} → MIN picks 2
- D's children: {9, 4} → MIN picks 4

MAX sees MIN will pick {3, 2, 4} → MAX picks 4 (move D is best)

Minimax value of A = 4

The Minimax Algorithm (Conceptually)

```

MINIMAX(node, isMaxPlayer):
    if node is terminal:
        return its utility value

    if isMaxPlayer:
        return MAX of MINIMAX(child, FALSE) for each child
    else:
        return MIN of MINIMAX(child, TRUE) for each child

```

Properties

Property	Minimax
Complete?	✓ Yes (in finite games)
Optimal?	✓ Yes (against optimal opponent)
Time	$O(b^m)$
Space	$O(bm)$

Problem: Chess has ~35 moves per position, game length ~80 moves.

Total nodes = $35^{80} \approx 10^{123}$. **Impossible to explore fully!**

3.12 Alpha-Beta Pruning

The Insight

"We don't need to explore every node. We can prune branches that cannot possibly affect the final decision."

Alpha-Beta prunes branches that are guaranteed to not be chosen.

The Two Values

- **α (alpha):** Best value MAX has found so far along the path (MAX's guarantee)
- **β (beta):** Best value MIN has found so far along the path (MIN's guarantee)

Pruning rule:

- If at a MIN node, and we find a value $\leq \alpha \rightarrow$ prune (MAX won't go here anyway)
- If at a MAX node, and we find a value $\geq \beta \rightarrow$ prune (MIN won't allow this anyway)

Worked Example

```

MAX:      A [ $\alpha=-\infty$ ,  $\beta=+\infty$ ]
          / \
MIN:     B   C
        /|  |\
       3 5  4 (...)
```

Exploring B:

- See 3 $\rightarrow \alpha$ becomes 3
- See 5 $\rightarrow \alpha$ becomes 5 $\rightarrow$ B's value = 5 for MAX

Back at A: $\alpha = 5$ (MAX knows it can get 5)

Exploring C:

- See 4 $\rightarrow$ MIN's value becomes 4
- $4 < \alpha$ ($4 < 5$) $\rightarrow$ **PRUNE** remaining children of C! MAX already has 5 from B. C will give at most 4. MAX won't pick C.

Efficiency Gain

Best case: Alpha-beta reduces the branching factor from b to $\sqrt{b}$

Meaning: instead of 35^{80} , we can explore $35^{40} \approx 10^{62}$

Still huge, but **dramatically smaller** — AI can now look twice as deep!

Modern chess engines combine alpha-beta with:

- Move ordering (explore best moves first)
- Transposition tables (don't revisit seen states)
- Iterative deepening → Stockfish can look 30+ moves ahead!

Math Intuition Corner

Heuristic Dominance

If $h_1(n) \geq h_2(n)$ for all n (and both are admissible):

- h_1 **dominates** h_2
- h_1 is a **better heuristic** (more informed)
- A^* with h_1 expands fewer nodes

This is a partial ordering of heuristics by quality.

The ideal (but unachievable) heuristic: $h^*(n) = \text{true cost to goal}$.

The Relaxed Problem Technique

How to construct admissible heuristics:

Solve a *relaxed* version of the problem (remove some constraints).

The optimal solution to the relaxed problem $\leq$ optimal solution to real problem.

Therefore: cost of relaxed solution = admissible heuristic!

8-puzzle example:

- **Real problem:** Tiles can only move to adjacent blank square
- **Relaxation 1:** Tiles can move anywhere → h_1 (misplaced tiles)
- **Relaxation 2:** Tiles can move to any adjacent square → h_2 (Manhattan distance)

Manhattan is less relaxed (fewer constraints removed) → it's a tighter, better bound.



Video Resources

Topic	Video	Why Watch
BFS & DFS visualized	Graph Traversals — CS Dojo	Side-by-side comparison
A* Search explained	A* Pathfinding — Sebastian Lague	BEST visual explanation
A* Interactive	Red Blob Games A*	Interactive — play with it!
Minimax algorithm	Minimax — Sebastian Lague	Brilliant game tree visualization
Alpha-Beta Pruning	Alpha-Beta Pruning — Patrick Winston MIT	From the master himself



Worked Example: 8-Puzzle with A*

Initial State:

```

|_|_|_|
|7|2|4|
|_|_|_|
|5|_|6|
|_|_|_|
|8|3|1|
|_|_|_|

```

Goal State:

```

|_|_|_|
|1|2|3|
|_|_|_|
|4|5|6|
|_|_|_|
|7|8|_|
|_|_|_|

```

Heuristic: Manhattan distance

Let's calculate h for the initial state:

Tile	Current	Goal	Distance
7	(0,0)	(2,0)	2
2	(0,1)	(0,1)	0
4	(0,2)	(1,1)	2
5	(1,0)	(1,0)	0
6	(1,2)	(1,2)	0
8	(2,0)	(2,1)	1
3	(2,1)	(0,2)	3
1	(2,2)	(0,0)	4

$h(\text{initial}) = 2+0+2+0+0+1+3+4 = \mathbf{12}$

$f(\text{initial}) = g(0) + h(12) = \mathbf{12}$

A* expands the node with lowest f — and eventually finds the optimal solution!

⚡ Exam-Ready Summary

All 4 Uninformed Search Algorithms

Algorithm	Complete	Optimal	Time	Space	Key Feature
BFS	✓	✓	$O(b^d)$	$O(b^d)$	Finds shortest path
DFS	✗	✗	$O(b^m)$	$O(bm)$	Very memory-efficient
DLS	✗	✗	$O(b^l)$	$O(bl)$	DFS with limit
IDDFS	✓	✓	$O(b^d)$	$O(bd)$	Best of both

Informed Search

Algorithm	Uses	Complete	Optimal
Hill Climbing	$h(n)$	✗	✗
Greedy BFS	$h(n)$	✗	✗
A*	$f(n)=g(n)+h(n)$	✓	✓ (if h admissible)

Adversarial Search

Algorithm	Idea	Key Optimization
Minimax	Both play optimally	—
Alpha-Beta	Same as Minimax	Prune branches

Alpha-Beta key rule: Prune when:

- At MIN node: value $\leq \alpha$ (MAX's current best)
- At MAX node: value $\geq \beta$ (MIN's current best)

✓ Chapter 3 Checklist

- ☐ Trace BFS and DFS on a given tree by hand
- ☐ Compare all 4 uninformed searches using the property table
- ☐ Explain what a heuristic is and what "admissible" means
- ☐ Calculate Manhattan distance for an 8-puzzle state
- ☐ Explain the $f(n) = g(n) + h(n)$ formula in your own words
- ☐ Trace the minimax algorithm on a given game tree
- ☐ Apply alpha-beta pruning and mark which nodes get pruned
- ☐ Explain why α - β doesn't change the minimax value

🔗 Navigation

← **Previous:** [Chapter 2 — Problem Solving](#)

→ **Next:** [Chapter 4 — Knowledge Representation](#)

 **Home:** [README](#)